

Task HD6

Name: Kha Anh Nguyen

Student ID: 103814796

Submission Date: 27th September 20225

Contents

Task HD6	1
Overview	2
Prototyping.....	2
1. Login view.....	2
2. Sign in view	3
3. Cart view	3
Code implementation	4
1. Views	4
2. Root App (app.js).....	6
3. Reusable components	7
4. Login, register, checkout, session storage	8
5. Currency Filter, computed, methods	10
6. Vuetify forms & validation and directive.....	10
7. Visual consistency	11
8. Data fetching	12
Web results	12
1. Large desktop view.....	12
2. Medium ipad view	15
3. Small phone view	17

Overview

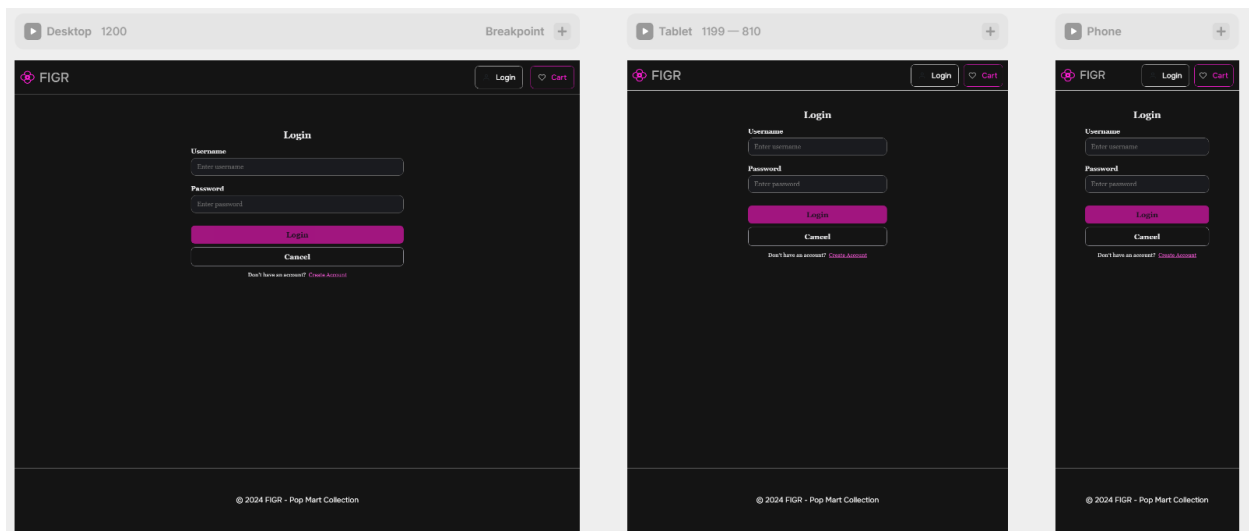
To enhance D6 work, I reorganized the app into a **modular structure** which is cleaner, and easier for me to track as the code grows. The **root app.js** holds the model (products, user, cart, totals) and allows **view** switching to components like Home, Cart, Checkout, and Profile via a **simple v-if condition** against **currentView** value. I also refactored repeated UI into **reusable** components (like custom button, product card, nav bar), added a **currency filter** for consistent prices, and configured app's dark theme with a reusable primary variable pink color for contrast.

I also made use of **pre-made Vuetify** elements (menu, form, validation), which helps keep the template small while ensures accessibility. I love to have a pretty UI, so I selected the images and constructed the data JSON myself. I hosted the JSON and images via Vercel to demonstrate fetching practice, with local JSON fetch as backup, then use **v-for** to loop through the product cards array efficiently. For accessibility, I added **labels** for form controls, search with **focus** states, and ensure the app remains **responsive** from phone to desktop. I've tried to maintain clear coding practices with comment block so future me can revise as well.

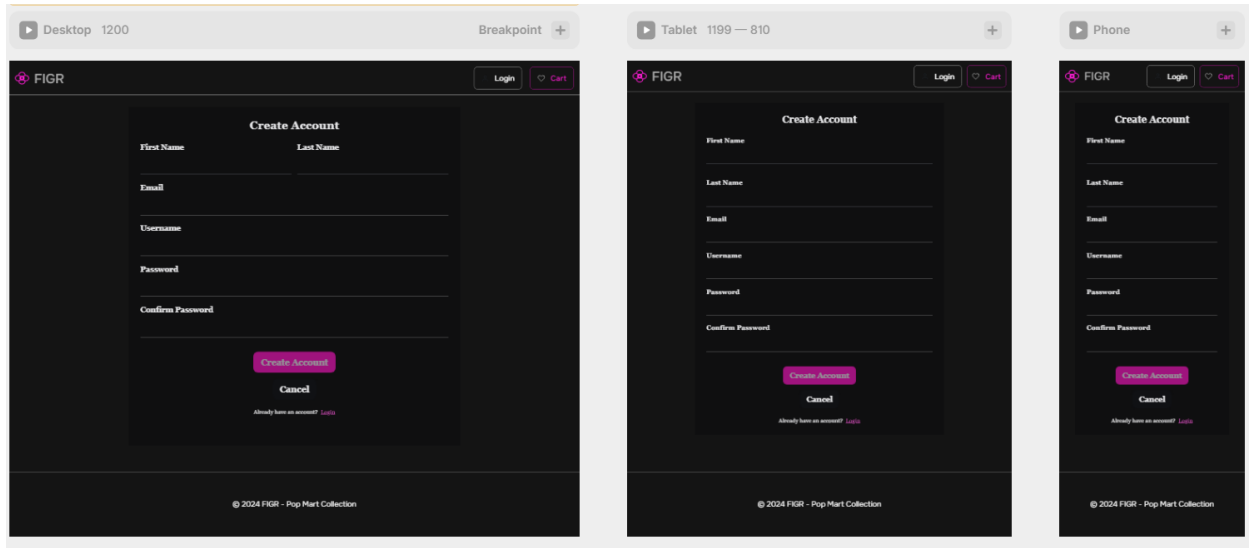
Prototyping

Again, I use Framer to help me with the wireframe and prototype some more components before I code:

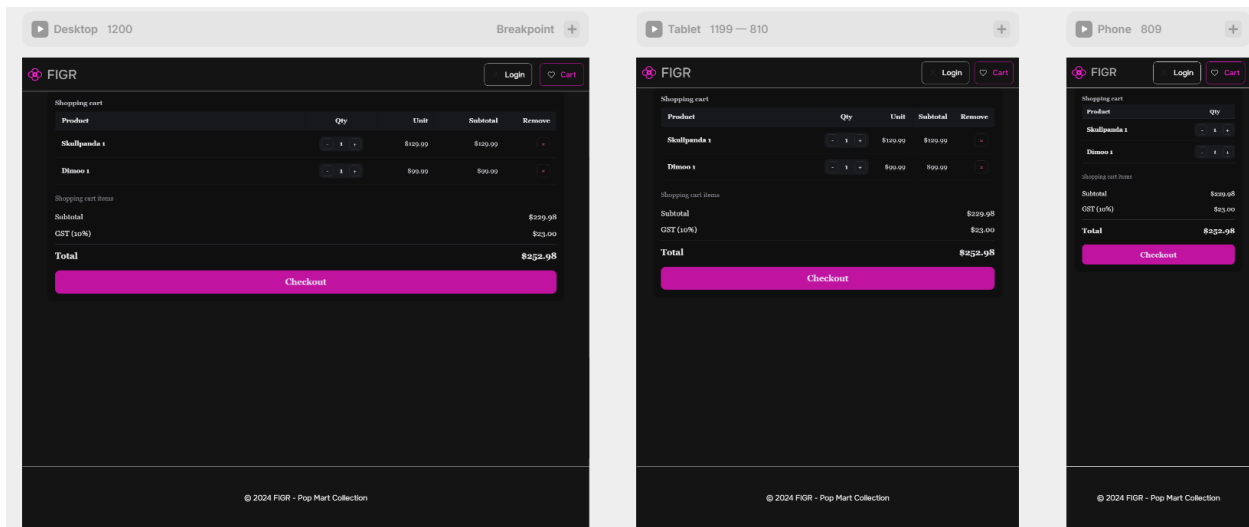
1. Login view



2. Sign in view



3. Cart view



For context, I implemented a modular Single Page Application (SPA) structure using conditional **v-if** view rendering. I intentionally avoided the complexity of a Vue Router setup, since the app is small-scale and self-contained. The condition help me to easily switch views like Home, Cart, Checkout, etc., via a reactive **currentView** value.

The **state flow pattern** below is the core to my Vue's design, I use it for both views and button props & UI functionalities:

Flow	Direction	Mechanism
Data	Parent → Child	via props
Events	Child → Parent	via \$emit()

Eg: The children components (views, buttons, etc.) tell the parent (root app.js, view components) to navigate via \$emit, and the parent updates state.

As I designed the prototype beforehand, I found out that some UI elements, like **buttons**, would **repeat** across views like Cart, Login, Checkout, etc., each with slightly different purposes but similar design. To avoid rewriting logic and maintain a clean component architecture, I built a **CustomButton** component that can be easily **reused**. (detailed code will be in section 3. Reusable component and 7. Visual consistency where I coded 4 button design variations based on 1 base design and separate the functionalities props). Also, form validation patterns would repeat in Login/Register so I would pass the validation props state globally at root App.js instead of locally in the component view to reuse them.

Code implementation

1. Views

Like the mockup, at **index.html**, parent listens and renders (note that i omitted the specialized methods here)

```
<v-app class="d-flex flex-column min-vh-100">
  <!-- Header with navigation -->
  <header class="mb-2">
    <nav-bar
      @go-to-home="goToHome"
      @go-to-profile="goToProfile"
      @go-to-login="goToLogin"
      @go-to-cart="goToCart"
      @logout="logout">
    </nav-bar>
```

```
</header>

<!-- Main content -->
<main id="main" class="my-10 justify-content-center flex-grow-1">
  <!-- Home View -->
  <home-view v-if="currentView==='home'"> </home-view>

  <!-- Cart View -->
  <cart-view v-if="currentView==='cart'"
    @go-to-checkout="goToCheckout"
    @go-to-login="goToLogin">
  </cart-view>

  <!-- Login View -->
  <login-view
    v-if="currentView==='login'"
    @go-to-home="goToHome"
    @go-to-register="goToRegister">
  </login-view>

  <!-- Register View -->
  <register-view
    v-if="currentView==='register'"
    @go-to-home="goToHome"
    @go-to-login="goToLogin">
  </register-view>

  <!-- Profile View -->
  <profile-view
    v-if="currentView==='profile'"
  </profile-view>

  <!-- Checkout View -->
  <checkout-view
    v-if="currentView==='checkout'"
    @go-to-cart="goToCart"
    @go-to-login="goToLogin">
  </checkout-view>
</main>

<!-- Footer -->
<footer></footer>
</v-app>
```

Root navigation methods:

```
methods: {  
  // Navigation methods for components  
  goToHome() { this.currentView = 'home'; },  
  goToCart() { this.currentView = 'cart'; },  
  goToCheckout() { this.currentView = 'checkout'; },  
  goToLogin() { this.currentView = 'login'; },  
  goToRegister() { this.currentView = 'register'; },  
  goToProfile() { this.currentView = 'profile'; },  
}
```

2. Root App (app.js)

The **app.js** acts as the **root** file, which maintains the **global** reactive **state** of the whole **application** such as current view, filters/sort, products, cart, user session, users DB, and order history. I also define **Vuetify** validation rules here so **Login/Register** can **reuse** them. Otherwise, **each view component** can handle its own **local state** and only parse the global, if necessary, to keep concerns separated.

```
new Vue({  
  el: '#app',  
  vuetify: vuetify,  
  
  data: {  
    currentView: 'home',  
    filters: { q: '', type: 'All' },  
    sortOrder: 'normal', // sort selector (default to no sorting)  
    categories: ['All', 'Skullpanda', 'Hirono', 'Dimoo'], // Category filter  
  },  
  options:  
    products: [],  
    cart: [],  
  
    // User authentication  
    user: null,  
  
    users: [], // Store registered users  
    orderHistory: [], // Order history  
  
    // and validation rules like name, password, email, etc.  
}
```

Totals, login gate, and cart count are computed so the UI updates instantly:

```
computed: {  
  // Ref: C3.3 compute  
  cartCount() {  
    return this.cart.reduce((a,c) => {return a+c.qty},0);  
  },  
  total() {  
    return this.cart.reduce((sum,c) => { return sum + c.price*c.qty; }, 0);  
  },  
  gst() {  
    return this.total * 0.1;  
  },  
  grandTotal(){ return this.total + this.gst; },  
  
  // User authentication computed  
  isLoggedIn(){ return !!this.user; },  
  canCheckout(){ return this.isLoggedIn && this.cart.length > 0; },  
},
```

3. Reusable components

For decomposition, I created a **custom-button** focusing on **UI** and **emits click**. Parents own the behavior, so I can reuse the same button across login, cart, and checkout without mixing logic into UI. This keeps the child stateless and the parent in control.

```
Vue.component('custom-button', {  
  template: `  
    <button  
      :class="buttonClasses"  
      :disabled="disabled"  
      @click="$emit('click', $event)"  
    >  
      <slot></slot>  
    </button>  
  `,  
})
```

Same as D6, the product display card is reused throughout the app using the product-card component. I loop through products using v-for, which renders efficiently due to a **unique :key binding**. Since it was mentioned in the D6 report, I am not mentioning the code here.

4. Login, register, checkout, session storage

I implemented a lightweight auth model using sessionStorage. Register writes to a simple in-memory “users” list and persists it; login loads it and writes the active user to storage. The checkout is gated by a computed canCheckout which requires both login and a non-empty cart.

Auth state + guards:

```
// User authentication computed at root app.js
isLoggedIn(){ return !!this.user; },
canCheckout(){ return this.isLoggedIn && this.cart.length > 0; },
```

Then for login / logout persistence:

```
// User account with sessionStorage persistence :)
// https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage
login(creds){
  if (creds.username && creds.password) {
    // Check if user exists in registered users
    var foundUser = this.users.find((u_db) =>{ return u_db.username ===
creds.username;});

    if (foundUser) {
      // User exists - create session
      this.user = {
        username: foundUser.username,
        name: foundUser.name,
        email: foundUser.email,
        registrationDate: foundUser.registrationDate,
        loginTime: new Date().toISOString()
      };

      this.saveUser(); // Save current user session
      this.currentView = 'home';
    } else {
      alert('Username not found. Please register first.');
```

```

logout(){
  this.user = null;
  this.saveUser(); // Save current user session
  this.currentView = 'home';
},

```

Session storage and app start loading:

```

methods: {

  // Generic data persistence
  saveToStorage(key, data) {
    sessionStorage.setItem(key, JSON.stringify(data));
  },

  loadFromStorage(key, defaultValue = []) {
    var saved = sessionStorage.getItem(key);
    return saved ? JSON.parse(saved) : defaultValue;
  },

  // User database to store & load multiple user accounts (when register)
  saveUsers() { this.saveToStorage('hd6_users', this.users); },
  loadUsers() { this.users = this.loadFromStorage('hd6_users', []); },

  saveOrderHistory() { this.saveToStorage('hd6_orders', this.orderHistory); },
  loadOrderHistory() { this.orderHistory = this.loadFromStorage('hd6_orders',
[]); },

  saveCart() { this.saveToStorage('cart', this.cart); },
  loadCart() { this.cart = this.loadFromStorage('cart', []); }

},

```

```

// Load all user-related data when app is created
created(){
  // Load all user-related data
  this.loadUser();
  this.loadUsers();
  this.loadOrderHistory();
  this.loadCart();
},

```

5. Currency Filter, computed, methods

Computed vs methods (quick note)

- I keep derived values in computed so the UI updates instantly and efficiently (totals, GST, grand total).
- I keep user actions in methods (add/remove, inc/dec, navigation).

This is also mentioned in D6, so I will move on with more new features explanation.

6. Vuetify forms & validation and directive

Same as D6, I used Vuetify/Bootstrap grid to keep the layout **responsive** from mobile to desktop, and product cards change per row based on breakpoints.

In addition, I switched to **Vuetify's pre-made form** validation, and use Vuetify v-form + v-text-field with **shared rules** passed from root to keep templates lean and accessible.

Root exposes the **rule arrays** as **props** to the Register **view**:

```
<!-- Register View -->
<register-view
  v-if="currentView==='register'"
  :first-name-rules="firstNameRules"
  :last-name-rules="lastNameRules"
  :email-rules="emailRules"
  :username-rules="usernameRules"
  :password-rules="passwordRules"
  @register="register($event)"
  @go-to-home="goToHome"
  @go-to-login="goToLogin">
</register-view>
```

I also add a tiny custom **directive** to **auto-focus** the first input (nice UX for forms):

```
// Customw directive
Vue.directive('focus', {
  inserted (el) { try { el.focus(); } catch(e){} }
});
```

7. Visual consistency

I configured Vuetify dark theme + a pink primary so the app has a consistent theme like Bootstrap and Vue.

```
// Initialize Vuetify
Vue.use(Vuetify);
const vuetify = new Vuetify({
  theme: {
    dark: true,
    themes: {
      dark: {
        primary: '#FF1FD6',
        background: '#141414',
        surface: '#1a1a1a'
      }
    }
  }
});
```

The custom-button variants ensure consistent button visuals across the app.

```
/* 1. Primary Pink - Solid pink background, white text */
.custom-btn--primary-pink {
  background-color: #FF1FD6;
  color: white;
}

/* 2. Outline Primary Pink - Pink outline, pink text, dark background */
.custom-btn--outline-primary-pink {
  background-color: transparent;
  color: #FF1FD6;
  border: 1px solid #FF1FD6;
}

/* 3. Secondary Dark - Solid dark background, white text */
.custom-btn--secondary-dark {
  background-color: #141414;
  color: white;
}

/* 4. Outline Secondary Dark - White outline, white text, dark background */
.custom-btn--outline-secondary-dark {
  background-color: transparent;
  color: white;
}
```

```
border: 1px solid #ADADAD;
}
```

8. Data fetching

To demonstrate dynamic content loading, I hosted my JSON product list and images on Vercel. If the remote fetch fails (e.g., due to picky cors issues), the app will fall back to loading a local JSON file.

The fetching is called when the app is instantiated.

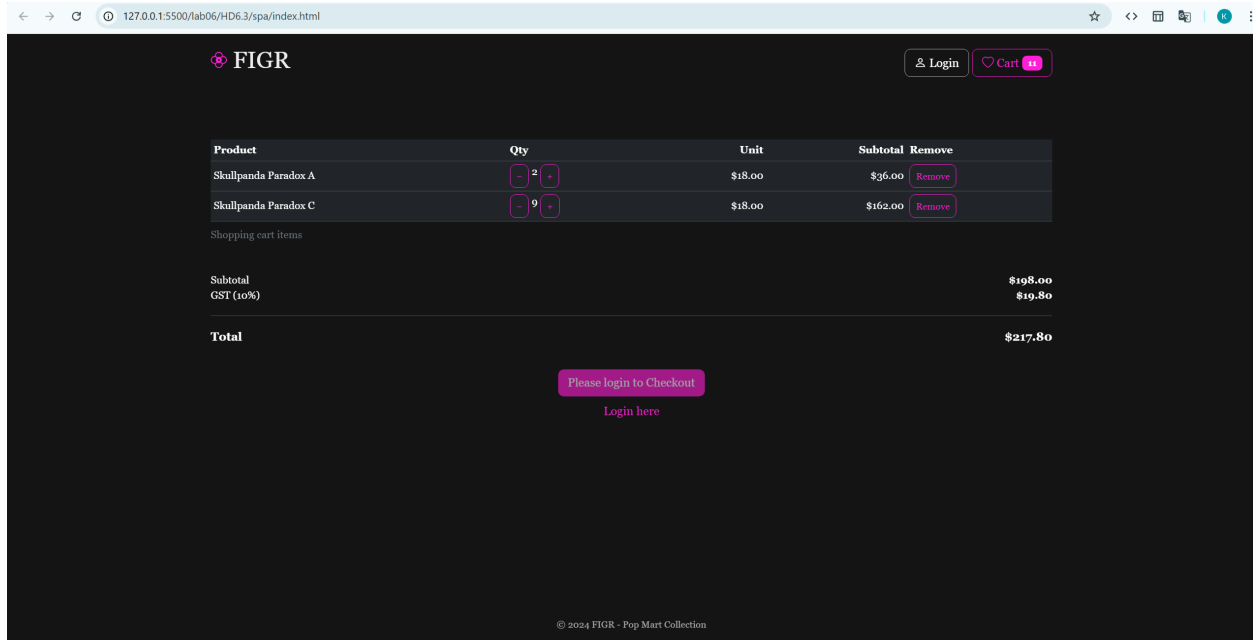
```
// Load all user-related data when app is created
created(){
  var self = this;
  // Load products from GitHub repository
  fetch('https://raw.githubusercontent.com/Leissha/popmart-
collection/main/products.json').then((r) => { return r.json(); }).then((list) =>
{
  self.products = list;
}).catch((error) => {
  console.error('Failed to load products from GitHub:', error);
  // Fallback to local JSON if GitHub fails (Swin cors protection)
  fetch('products.json').then((r) => { return r.json(); }).then((list) => {
    self.products = list;
  });
});
});
}
```

Web results

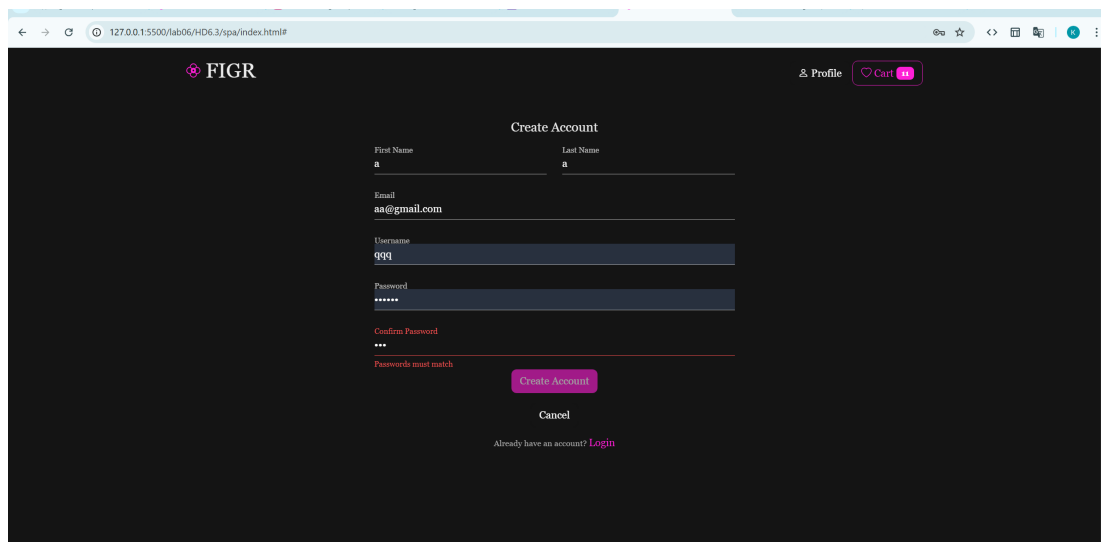
1. Large desktop view

1.1 Cart

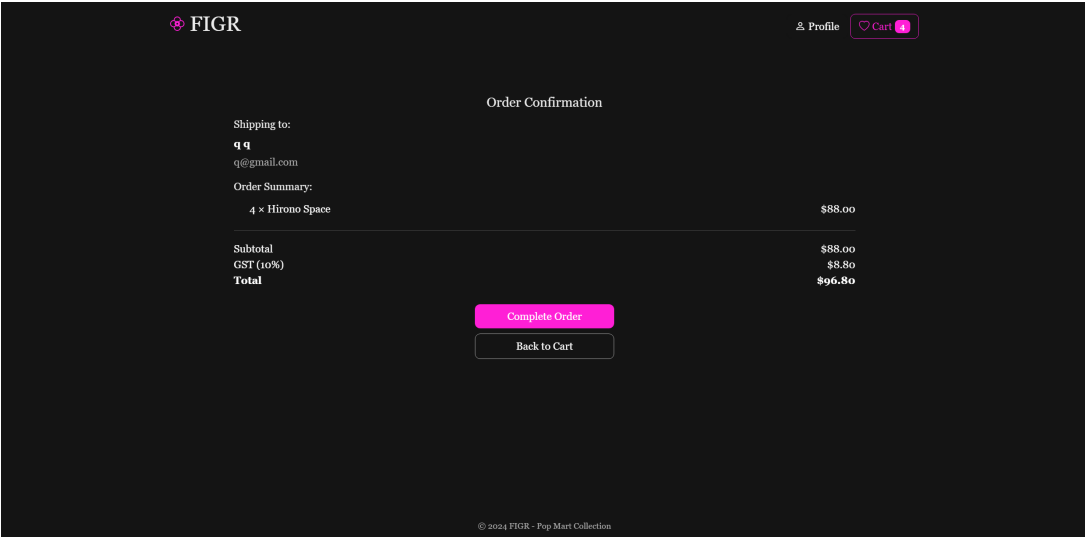
User cannot check out if has not logged in



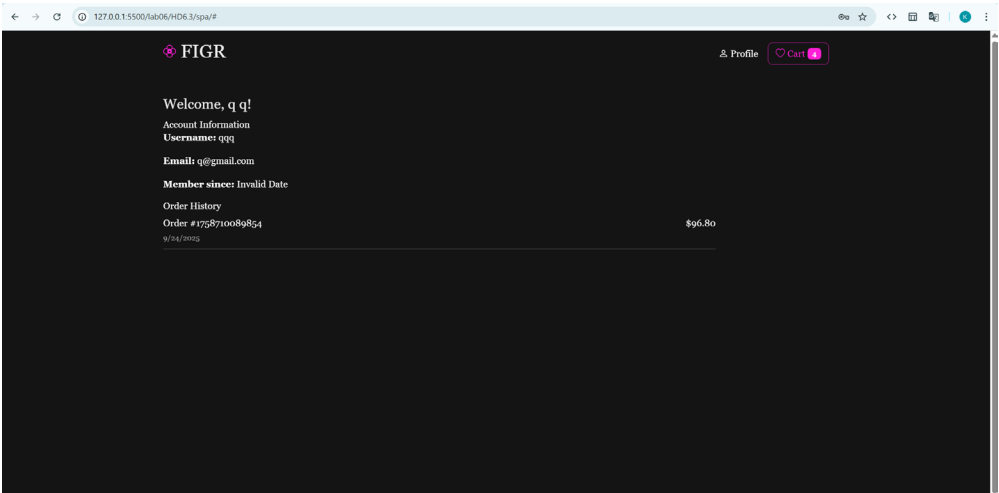
1.2. Vuetify sign in validation form



1.3. Checkout

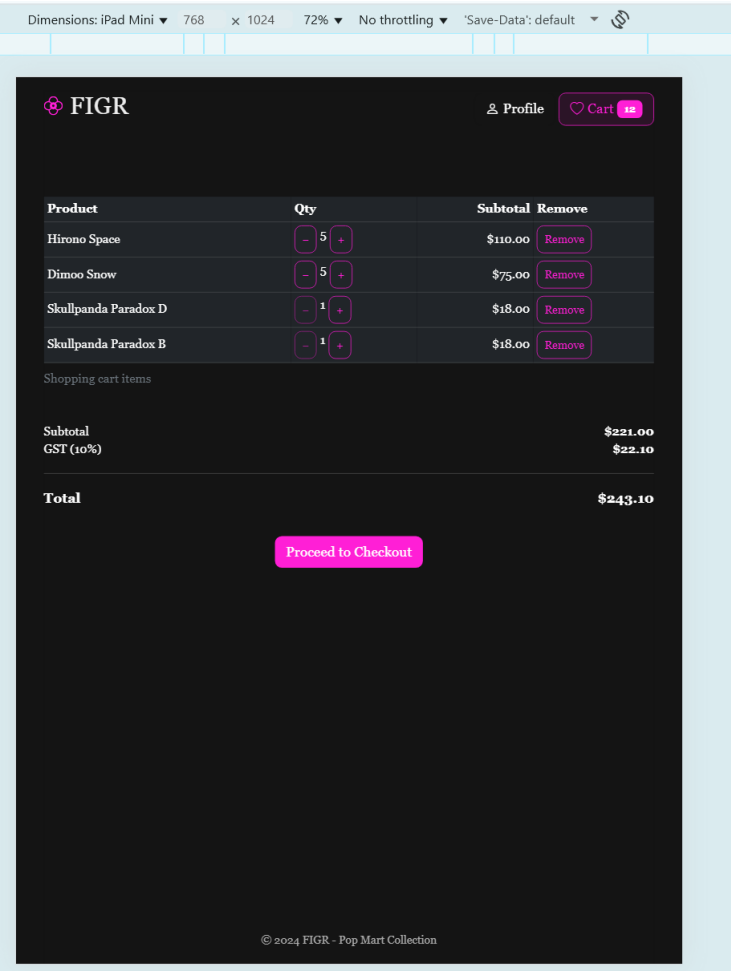


1.4. User session storage profile

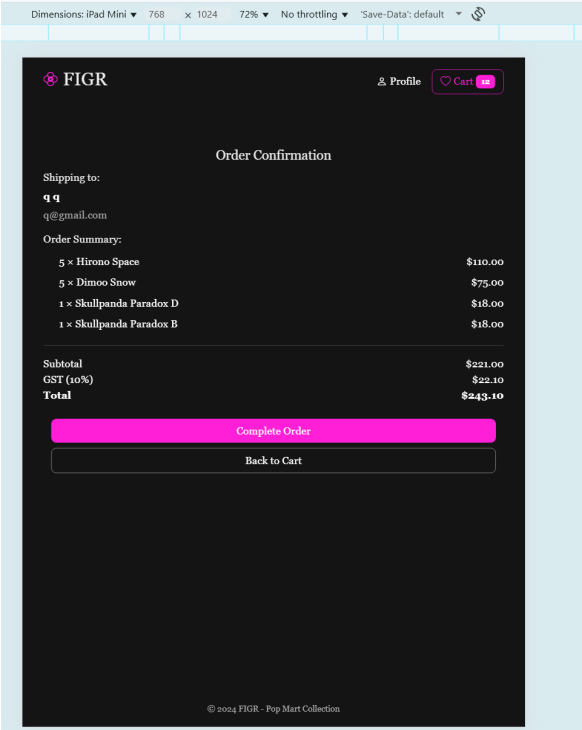


2. Medium ipad view

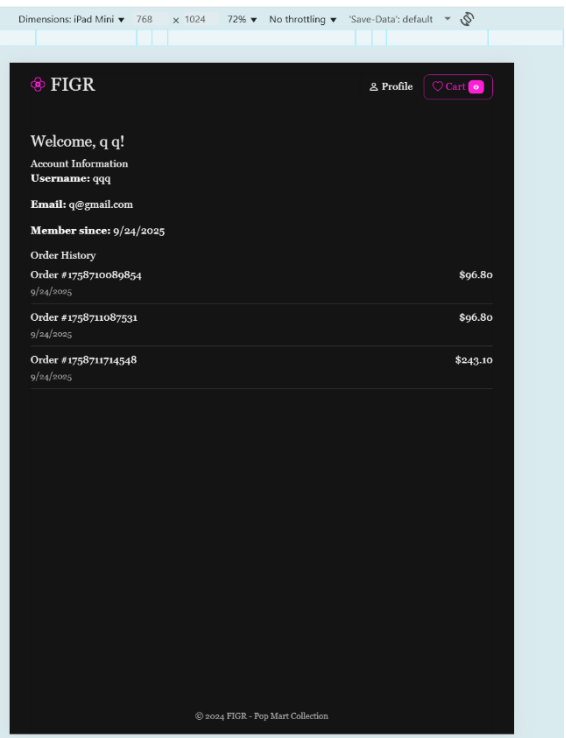
2.1 Cart



2.2. Checkout

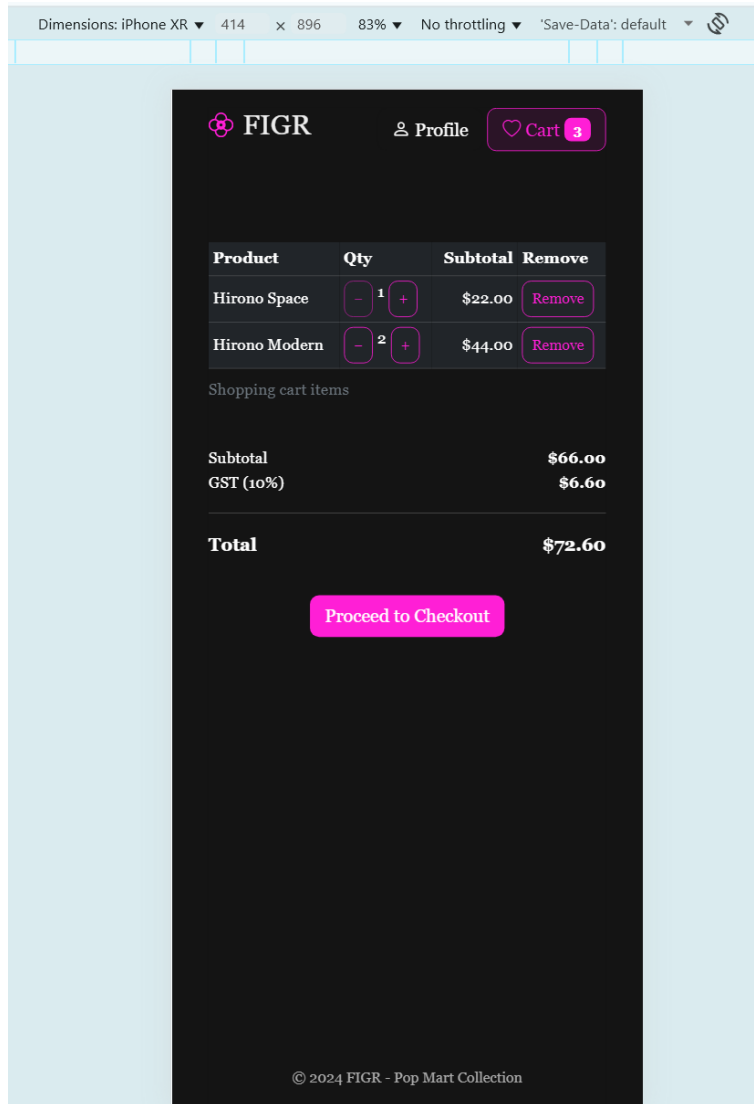


2.3. Profile

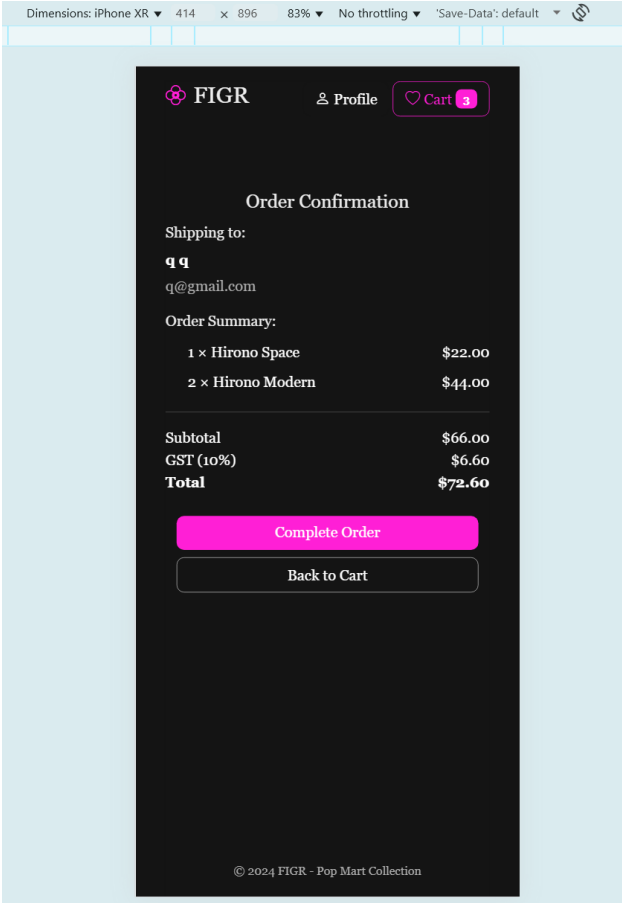


3. Small phone view

3.1 Cart



3.2 Checkout



3.3 Profile

